

PROJET 28

Serveur de Parametres (Parameter Server)

Architecture distribuee asynchrone pour l'entrainement de modeles ML

P28

Type de document :

Cahier des Charges

Module :

Systemes Distribues

Annee :

2024 - 2025

Niveau :

4eme annee IIR — Semestre 7

REALISE PAR

Anas ELAYATTI

Steven KABRE

1. Contexte et Problematique

L'entrainement de modeles de Machine Learning sur de grands volumes de donnees est une problematique centrale. Sur une seule machine, traiter des millions d'exemples prend plusieurs heures. Le Projet 28 repond a ce defi en implementant une **architecture Parameter Server** : un systeme distribue ou plusieurs workers collaborent de facon **asynchrone** pour acclereler l'entrainement, coordonnees par un serveur central de parametres.

1.1 Probleme resolu

Critere	1 Machine (sequentiel)	Parameter Server (3 workers)
Duree par epoque	~60 minutes	~20 minutes
Tolerance pannes	Nulle	Elevee (1 worker peut tomber)
Scalabilite	Limitee par le CPU	Horizontale (ajouter workers)
Coherence	Parfaite	Eventuelle (staleness ok)

1.2 Cas d'usage reel

Cette architecture est utilisee en production par **Google** (DistBelief, TensorFlow Distributed), **Meta** (PyTorch Distributed) et **Microsoft** (Azure ML) pour entrainer des modeles de milliards de parametres sur des clusters de milliers de machines.

2. Objectifs du Projet

Concevoir et implementer un systeme distribue complet de type Parameter Server, demontrant les concepts fondamentaux : parallelisme, coherence eventuelle, tolerance aux pannes et scalabilite.

2.1 Objectifs fonctionnels

- **Serveur de Parametres central** : stocker les poids W , recevoir et appliquer les gradients, renvoyer les poids a jour.
- **Workers distribues** : charger un sous-ensemble des donnees, calculer les gradients en parallele, communiquer avec le serveur de facon asynchrone.
- **Modele ML** : regression logistique binaire avec descente de gradient stochastique.
- **Convergence** : precision > 90% sur un dataset de classification binaire.

2.2 Objectifs non-fonctionnels

- **Asynchronisme** : aucun worker n'attend les autres.
- **Thread-safety** : mises a jour protegees par un verrou (mutex).
- **Robustesse** : la panne d'un worker ne doit pas interrompre l'entrainement.
- **Observabilite** : loguer chaque gradient recu, le staleness, la version du modele.

3. Architecture Technique

L'architecture suit le patron **maître-esclaves asynchrone**. Le serveur est le noeud central, les workers sont les noeuds de calcul. La communication suit un protocole **Push/Pull** en deux phases.

3.1 Composants principaux

ParameterServer	Serveur central. Stocke les poids W . Recoit les gradients (push) et renvoie les poids (pull). Protege par <code>threading.Lock()</code> pour la thread-safety.
Worker	Thread independant (<code>threading.Thread</code>). Recoit un sous-ensemble des donnees. Boucle : Pull poids → Calcul gradient → Push gradient. Avance sans attendre.
Protocole Push/Pull	PULL : le worker demande les poids actuels et leur version. PUSH : le worker envoie son gradient. Le serveur applique $W = W - lr * \text{gradient}$.

3.2 Flux de donnees

Etape	Acteur	Action	Direction
1 — Init	Serveur	Genere W aleatoires, decoupe dataset	→ Workers
2 — Pull	Worker i	Demande les poids actuels (version v)	$W \rightarrow Srv$
3 — Gradient	Worker i	Calcule gradient sur son mini-batch local	Local
4 — Push	Worker i	Envoie gradient au serveur	$W \rightarrow Srv$
5 — Update	Serveur	$W = W - lr * \text{gradient}$, version++	Interne
6 — Boucle	Tous	Repeter 2-5 jusqu'a convergence	Loop

4. Algorithmes et Formules Cles

4.1 Modele : Regression Logistique

Prediction	$p = \text{sigmoid}(X \cdot W) = 1 / (1 + \exp(-X \cdot W))$
Perte (Cross-Entropy)	$L = -[y * \log(p) + (1-y) * \log(1-p)]$
Gradient	$dL/dW = (1/n) * X^T \cdot (p - y)$
Mise a jour poids	$W(t+1) = W(t) - \alpha * \text{gradient}$

4.2 Staleness

Le staleness mesure le decalage entre la version des poids utilisee par un worker pour calculer son gradient et la version actuelle du serveur.

$$\text{staleness} = \text{version_serveur} - \text{version_locale_worker}$$

Staleness	Signification	Impact
0	Gradient frais — poids parfaitement a jour	Optimal
1 - 2	Leger decalage	Negligeable
3 - 10	Decalage modere	Ralentissement possible
> 20	Worker tres lent	Risque de divergence

5. Analyse CAP et Compromis

Le theoreme CAP (Brewer, 2000) stipule qu'un systeme distribue ne peut garantir simultanement que deux des trois proprietes : Coherence, Disponibilite, Tolerance aux partitions.

Propriete	Definition	Notre systeme
C — Coherence	Tous les noeuds voient la meme version	Partielle (staleness)
A — Disponibilite	Chaque requete recoit une reponse	OUI
P — Partition	Fonctionne malgre coupures reseau	OUI

Notre systeme est de type **AP** (Available + Partition-tolerant). On sacrifie la coherence forte au profit de la vitesse. Une coherence eventuelle suffit a la convergence ML.

5.1 Strategies de synchronisation

Strategie	Description	Avantages	Inconvenients
ASP (notre choix)	Aucune synchro	Vitesse maximale	Staleness non borne
SSP	Attend si staleness > seuil s	Compromis equilibre	Parametrage seuil
BSP	Synchro a chaque iteration	Coherence parfaite	Vulnerable straggler

6. Livrables Attendus (GitHub)

Livrable	Contenu	Fichier
Code source	Classes ParameterServer et Worker	parameter_server.py
Script entraînement	Donnees, entraînement, graphiques	train.py
Graphiques	Convergence, poids, architecture	resultats/*.png
Rapport Word	Partie theorique + pratique completes	Rapport_Projet28.docx
Cahier des charges	Ce document	CahierCharges_P28.pdf

7. Resultats Obtenus

Metrique	Valeur	Interpretation
Precision finale	94.5%	Excellent pour une regression logistique
Version du modele	v720	720 mises a jour en 15 epoques
Gradients par worker	240 chacun	Equilibre parfait entre les 3 workers
Temps d'entraînement	0.9 sec	3 workers Python threads
Staleness moyen	0.5	Tres faible — systeme bien equilibre
Erreur sur poids DW	~0.50	Poids appris proches des vrais poids

Le systeme a ete teste avec 1 a 5 workers, differents taux d'apprentissage (0.001 a 0.5), et avec simulation de pannes worker. Dans tous les cas, la convergence est maintenue, confirmant la robustesse de l'architecture.

8. Guide d'Utilisation

8.1 Installation et lancement

```
pip install numpy matplotlib
```

```
python train.py
```

8.2 Parametres configurables

Parametre	Defaut	Description
n_workers	3	Nombre de workers paralleles
n_epochs	15	Passages complets sur les donnees
batch_size	32	Taille du mini-batch par iteration

lr (alpha)	0.05	Taux d'apprentissage
n_samples	1500	Nombre total d'exemples

9. Conclusion

Le Projet 28 demontre concretement le fonctionnement d'un **Parameter Server** — architecture pilier de l'entrainement distribue de modeles ML a grande echelle. Les concepts cles implementes sont :

- **Asynchronisme** : chaque worker avance independamment, maximisant l'utilisation des ressources.
- **Thread-safety** : le verrou mutex garantit l'integrite des poids lors des mises a jour concurrentes.
- **Compromis CAP** : choix AP avec coherence eventuelle — optimal pour le ML distribue.
- **Robustesse** : la panne d'un worker ne casse pas le systeme.
- **Scalabilite** : ajouter des workers ameliore precision et vitesse.

Anas ELAYATTI
EMSI Marrakech — IIR S7

Steven KABRE
EMSI Marrakech — IIR S7